

How to Execute Multiple Tasks in C#

In this article, I will discuss **How to Execute Multiple Tasks in C# using the WhenAll Method** with Examples. Please read our previous article, which discusses [How to Return a Value from a Task in C#](#) with Examples.

How to Execute Multiple Tasks in C#?

So far, we have been executing one task at a time, but we sometimes have many tasks we want to execute simultaneously. We can do that with **Task.WhenAll** method. With **Task.WhenAll** method, we can have a list of tasks, and all the tasks will be executed concurrently. When all of the tasks are finished, we will be able to continue the execution of a method.

Example to Understand Task.WhenAll Method:

Let us understand how to execute multiple tasks concurrently using the Task.WhenAll method in C#. We will do an example where we want to process multiple credit cards.

We are going to use the following CreditCard class in our example. The following CreditCard class has two properties, i.e., CardNumber and Name, and one static method, i.e., GenerateCreditCards, to generate a collection of CreditCard. The GenerateCreditCards method takes one integer number as a parameter and then creates a collection of that many numbers of credit cards and returns that collection.

```
public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
```

Next, we need to create one async method to process the credit cards. For this, we are creating the following ProcessCard Async method. This method takes the CreditCard as an input parameter and processes that credit card. Here, you can make any API call to process the Credit Card. But for simplicity, we just delay the execution for 1 second using the Task.Delay Asynchronous Method and then print the message that the credit is processed and return a string containing the processed credit card information for future use if required.

```

public static async Task<string> ProcessCard(CreditCard creditCard)
{
    await Task.Delay(1000);
    string message = $"Credit Card Number: {creditCard.CardNumber} Name: {creditCard.Name} Processed";
    Console.WriteLine($"Credit Card Number: {creditCard.CardNumber} Processed");
    return message;
}

```

Next, we are creating another asynchronous method to execute multiple tasks concurrently. For this purpose, we are creating the following **ProcessCreditCards** async method. This method takes the collection of cards that we want to be processed. Then, the `ForEach` loop processes the cards individually by calling the `ProcessCard` async method. While calling the `ProcessCard` async method, we are not using the `await` operator. The return type of `ProcessCard` is `Task<string>`. So here, I have created a collection of type `Task<string>`, i.e., `List< Task<string>>` tasks, to store the response coming from the `ProcessCard` method. Next, we call the `Task.WhenAll` method by passing that `Task<string>` collection. To check the time, here we use a stopwatch showing the time the `WhenAll` method takes to process all the credit cards.

```

public static async void ProcessCreditCards(List<CreditCard> creditCards)
{
    var stopwatch = new Stopwatch();
    stopwatch.Start();
    var tasks = new List<Task<string>>();
    foreach (var creditCard in creditCards)
    {
        var response = ProcessCard(creditCard);
        tasks.Add(response);
    }
    await Task.WhenAll(tasks);
    stopwatch.Stop();
    Console.WriteLine($"Processing of {creditCards.Count} Credit Cards Done in {stopwatch.ElapsedMilliseconds / 1000.0} Seconds");
}

```

Please note the following statement:

1. **`await Task.WhenAll(tasks);`** This statement tells that there is a list of tasks. Please wait for all of the tasks to be done before continuing with the execution of this method, and all of the tasks will be run simultaneously. As tasks contain 10 entries, all these 10 tasks are to be executed simultaneously.

Next, Modify the `Main` method as follows. From the main method, we call the static `GenerateCreditCards` method of the `CreditCard` class by passing an integer number, i.e., 10, as an argument. This `GenerateCreditCards` method will return a collection of 10 Credit Cards. And then, we are calling the `ProcessCreditCards` by passing that credit card collection as an argument.

```

static void Main(string[] args)
{
    Console.WriteLine($"Main Thread Started");
    List<CreditCard> creditCards = CreditCard.GenerateCreditCards(10);
    Console.WriteLine($"Credit Card Generated : {creditCards.Count}");
}

```

```

        ProcessCreditCards(creditCards);
        Console.WriteLine($"Main Thread Completed");
        Console.ReadKey();
    }

```

Complete Example code:

Whatever we discussed as of now, everything is put in the below example.

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Main Thread Started");
            List<CreditCard> creditCards =
CreditCard.GenerateCreditCards(10);
            Console.WriteLine($"Credit Card Generated :
{creditCards.Count}");

            ProcessCreditCards(creditCards);

            Console.WriteLine($"Main Thread Completed");
            Console.ReadKey();
        }
        public static async void ProcessCreditCards(List<CreditCard>
creditCards)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            var tasks = new List<Task<string>>();
            //Processing the creditCards using foreach loop
            foreach (var creditCard in creditCards)
            {
                var response = ProcessCard(creditCard);
                tasks.Add(response);
            }
            //It will execute all the tasks concurrently
            await Task.WhenAll(tasks);
            stopwatch.Stop();
            Console.WriteLine($"Processing of {creditCards.Count} Credit
Cards Done in {stopwatch.ElapsedMilliseconds/1000.0} Seconds");
            //foreach(var item in tasks)
            //{
            //    Console.WriteLine(item.Result);

```

```

        //}
    }

    public static async Task<string> ProcessCard(CreditCard creditCard)
    {
        //Here we can do any API Call to Process the Credit Card
        //But for simplicity we are just delaying the execution for 1
second
        await Task.Delay(1000);
        string message = $"Credit Card Number: {creditCard.CardNumber}
Name: {creditCard.Name} Processed";
        Console.WriteLine($"Credit Card Number: {creditCard.CardNumber}
Processed");
        return message;
    }
}

public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
}

```

Output:

```
Main Thread Started
Credit Card Generated : 10
Main Thread Completed
Credit Card Number: 100000006 Processed
Credit Card Number: 100000008 Processed
Credit Card Number: 100000009 Processed
Credit Card Number: 100000000 Processed
Credit Card Number: 100000005 Processed
Credit Card Number: 100000002 Processed
Credit Card Number: 100000003 Processed
Credit Card Number: 100000007 Processed
Credit Card Number: 100000004 Processed
Credit Card Number: 100000001 Processed
Processing of 10 Credit Cards Done in 1.089 Seconds
```

You can see it is taking little more than 1 second to process all the Credit Cards. One more point: when we are executing multiple tasks concurrently, you can never predict the order of execution. Now, let us observe the output. If you remember, within the ProcessCard method, we delayed the execution for one second. But after that, as we execute multiple tasks using Task.WhenAll method, all the tasks execution is completed within little more than 1 second. This is because of Task.WhenAll method executes all the tasks concurrently, drastically improving our application's performance.

Execution without Task.WhenAll Method in C#:

Now, let us execute the same application without using Task.WhenAll and observe how long it takes to process 10 credit cards. Please modify the ProcessCreditCards method as follows. Here, we remove the Task.WhenAll method and its related code. Here, we are using the await operator.

```
public static async void ProcessCreditCards(List<CreditCard> creditCards)
{
    var stopwatch = new Stopwatch();
    stopwatch.Start();
    foreach (var creditCard in creditCards)
    {
        var response = await ProcessCard(creditCard);
    }
    stopwatch.Stop();
    Console.WriteLine($"Processing of {creditCards.Count} Credit Cards Done
in {stopwatch.ElapsedMilliseconds / 1000.0} Seconds");
}
```

With the above changes in place, run the application and observe the output shown in the image below.

```
Main Thread Started
Credit Card Generated : 10
Main Thread Completed
Credit Card Number: 100000000 Processed
Credit Card Number: 100000001 Processed
Credit Card Number: 100000002 Processed
Credit Card Number: 100000003 Processed
Credit Card Number: 100000004 Processed
Credit Card Number: 100000005 Processed
Credit Card Number: 100000006 Processed
Credit Card Number: 100000007 Processed
Credit Card Number: 100000008 Processed
Credit Card Number: 100000009 Processed
Processing of 10 Credit Cards Done in 10.16 Seconds
```

It takes more than 10 seconds to process 10 credit cards compared to a little more than 1 second when using Task.WhenAll Method in C#. Now, I hope you understand when and how to use Task.WhenAll in C#.

Offloading the Current Thread – Task.Run Method in C#

Let us understand what you mean by Offloading the Current Thread in C# with an Example. Let us modify the example as follows. Now, we are trying to process **100000** credit cards. In the example below, we removed the statement that prints the credit card details on the console. Further, we have used a stopwatch to check how long the main thread takes.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            Console.WriteLine($"Main Thread Started");
            List<CreditCard> creditCards =
            CreditCard.GenerateCreditCards(100000);
            Console.WriteLine($"Credit Card Generated :
            {creditCards.Count}");

            ProcessCreditCards(creditCards);

            Console.WriteLine($"Main Thread Completed");
            stopwatch.Stop();
            Console.WriteLine($"Main Thread Execution Time
            {stopwatch.ElapsedMilliseconds / 1000.0} Seconds");
        }
    }
}
```

```

        Console.ReadKey();
    }
    public static async void ProcessCreditCards(List<CreditCard>
creditCards)
    {
        var stopwatch = new Stopwatch();
        stopwatch.Start();
        var tasks = new List<Task<string>>();

        foreach (var creditCard in creditCards)
        {
            var response = ProcessCard(creditCard);
            tasks.Add(response);
        }
        //It will execute all the tasks concurrently
        await Task.WhenAll(tasks);
        stopwatch.Stop();
        Console.WriteLine($"Processing of {creditCards.Count} Credit
Cards Done in {stopwatch.ElapsedMilliseconds/1000.0} Seconds");
    }

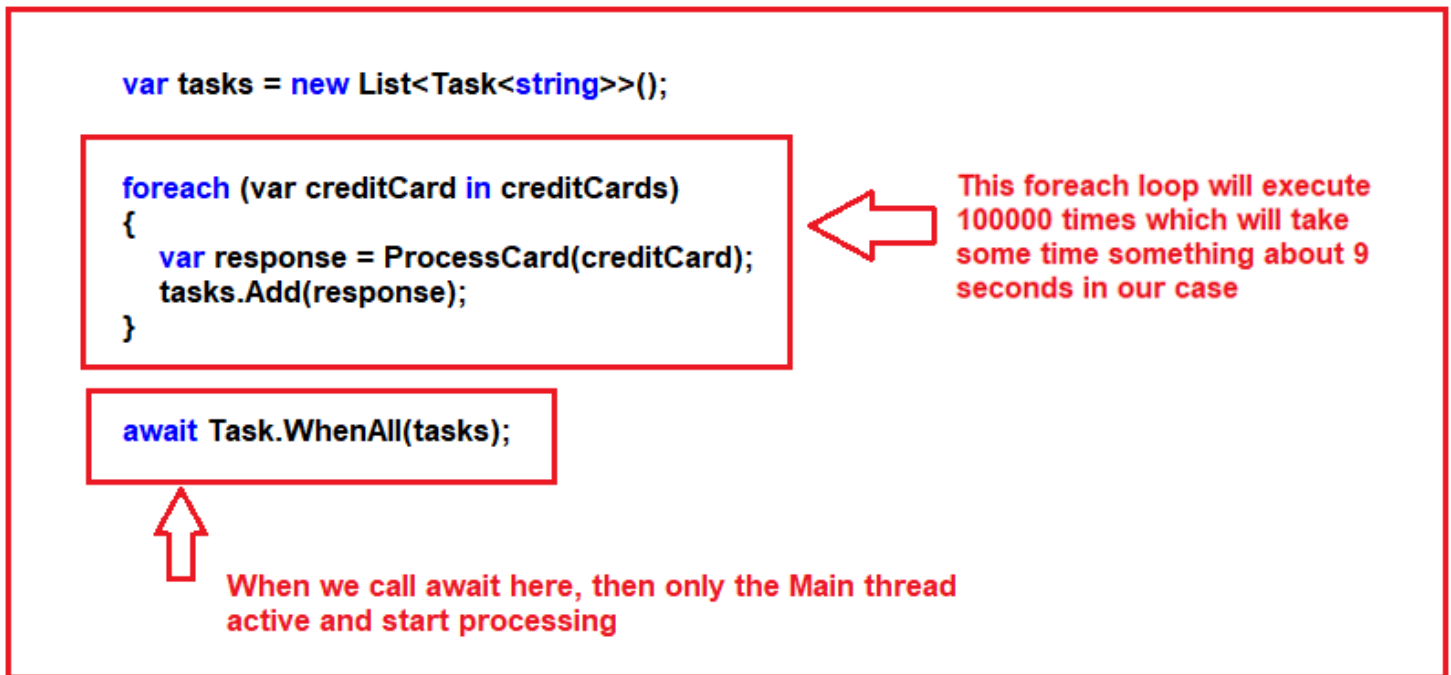
    public static async Task<string> ProcessCard(CreditCard creditCard)
    {
        await Task.Delay(1000);
        string message = $"Credit Card Number: {creditCard.CardNumber}
Name: {creditCard.Name} Processed";
        return message;
    }
}
public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
}

```

Output:

```
Main Thread Started  
Credit Card Generated : 100000  
Main Thread Completed  
Main Thread Execution Time 8.964 Seconds  
Processing of 100000 Credit Cards Done in 9.792 Seconds
```

You can see the Main thread taking approximately 9 seconds. Let us observe why. Please have a look at the below image. The following for each loop of our ProcessCreditCards method runs 100000 times, which will take some time, approximately 9 seconds. So, until the **await Task.WhenAll(tasks)** statement is called, our Main thread is frozen. As soon as we call, **await Task.WhenAll(tasks)** method, the thread is active and starts processing.



We don't want our Main thread to freeze for 9 seconds because one of the main reasons to use asynchronous programming in C# is to have a responsive UI. So, we don't want the UI or Main thread to be frozen.

How do we overcome the above problem?

In any way, we need to make the Main Thread available. For that, we can offload each loop to another thread using the Task.Run Asynchronous Method in C#. Let us see how. Please have a look at the below image. We need to use Task.Run method and using a delegate, we need to use the for each loop, further as Task.Run method is an asynchronous method, so we need to use the await operator, as shown in the image below.

```
foreach (var creditCard in creditCards)
{
    var response = ProcessCard(creditCard);
    tasks.Add(response);
}
```



```
await Task.Run(() =>
{
    foreach (var creditCard in creditCards)
    {
        var response = ProcessCard(creditCard);
        tasks.Add(response);
    }
});
```

With the above changes, the for-each loop will be executed by another thread, and as we use the await method before Task.Run so the main thread will be free and continue its execution. The complete example code is given below.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            Console.WriteLine($"Main Thread Started");
            List<CreditCard> creditCards =
            CreditCard.GenerateCreditCards(100000);
            Console.WriteLine($"Credit Card Generated :
            {creditCards.Count}");

            ProcessCreditCards(creditCards);

            Console.WriteLine($"Main Thread Completed");
            stopwatch.Start();
            Console.WriteLine($"Main Thread Execution Time
            {stopwatch.ElapsedMilliseconds / 1000.0} Seconds");
            Console.ReadKey();
        }
        public static async void ProcessCreditCards(List<CreditCard>
        creditCards)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
```

```

var tasks = new List<Task<string>>();
await Task.Run(() =>
{
    foreach (var creditCard in creditCards)
    {
        var response = ProcessCard(creditCard);
        tasks.Add(response);
    }
});

//It will execute all the tasks concurrently
await Task.WhenAll(tasks);
stopwatch.Stop();
Console.WriteLine($"Processing of {creditCards.Count} Credit
Cards Done in {stopwatch.ElapsedMilliseconds/1000.0} Seconds");
}

public static async Task<string> ProcessCard(CreditCard creditCard)
{
    await Task.Delay(1000);
    string message = $"Credit Card Number: {creditCard.CardNumber}
Name: {creditCard.Name} Processed";
    return message;
}
}
public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
}
}

```

With the above changes in place, now run the application and observe the output shown in the image below. Now, the main thread is not frozen and is completed in milliseconds.

```
Main Thread Started  
Credit Card Generated : 100000  
Main Thread Completed  
Main Thread Execution Time 0.102 Seconds  
Processing of 100000 Credit Cards Done in 9.919 Seconds
```

WhenAll Methods of Task Class in C#:

If you go to the definition of Task class, you will see four overloaded versions of this method available. They are as follows:

1. **WhenAll(IEnumerable<Task> tasks):** It Creates a task that will be completed when all of the Task objects in an enumerable collection have been completed. Here, the parameter tasks specify the tasks to wait on for completion. It returns a task that represents the completion of all supplied tasks.
2. **WhenAll<TResult>(params Task<TResult>[] tasks):** It creates a task that will be completed when all of the Task objects in an array have been completed. Here, the parameter tasks specify the tasks to wait on for completion. The Type parameter TResult specifies the type of the completed task. It returns a task that represents the completion of all supplied tasks.
3. **WhenAll<TResult>(IEnumerable<Task<TResult>> tasks):** It creates a task that will be completed when all of the Task objects in an enumerable collection have been completed. Here, the parameter tasks specify the tasks to wait on for completion. The Type parameter TResult specifies the type of the completed task. It returns a task that represents the completion of all supplied tasks.
4. **WhenAll(params Task[] tasks):** It creates a task that will be completed when all Task objects in an array have been completed. Here, the parameter tasks specify the tasks to wait on for completion. It returns a task that represents the completion of all supplied tasks.

In the next article, I will discuss [Limiting the Number of Concurrent Tasks in C#](#) with Examples. In this article, I explain **How to Execute Multiple Tasks using the WhenAll Method in C#** with Examples. I hope you enjoy this How to Execute Multiple Tasks using Task.WhenAll Method in C# article.